LESSON 03 · PYTEST · 50 MIN

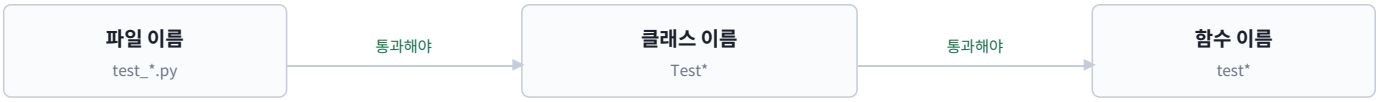
3교시 pytest 기초 — 수집과 실패 리포트

pytest는 이름으로 테스트를 찾습니다. 규칙에 안 맞으면 오류도 없이 조용히 지나갑니다. 수집 규칙을 하나씩 어겨 보고, 실패했을 때 리포트를 읽는 연습에 시간을 씁니다.

개념 12분 · 실습 28분 · 정리와 질문 10분

OVERVIEW

수집은 위에서 아래로 이어진다



위 단계가 막히면 아래는 **존재조차 확인되지 않습니다.** 실패가 아니라 수집이 안 된 것이라 터미널에는 **collected 0 items** 만 뜨고 오류는 나지 않습니다.

REFERENCE

이 시간에 쓰는 명령

명령	하는 일	언제 쓰나
`pytest`	현재 폴더 아래를 모두 뒤져 실행	기본 실행
`assert 값 == 기대값`	비교 실패 시 양쪽 값을 보여 준다	pytest가 assert를 다시 써 주기 때문
`pytest -v`	테스트 이름을 한 줄씩 보여 준다	무엇이 돌았는지 확인할 때
`pytest -k "키워드"`	이름으로 골라 실행	일부만 빠르게 돌릴 때
`pytest -x`	첫 실패에서 즉시 중단	원인 하나에 집중할 때
`pytest --lf`	직전에 실패한 것만 다시	고치는 중 반복 실행
`pytest.raises(예외)`	예외가 나는지 검증	실패해야 정상인 경우
`@pytest.mark.skip / xfail`	건너뛰기 / 실패 예상	이름을 바꿔 끄지 말 것

FILES

실습용 파일

cart.py

```
class EmptyCartError(Exception):
    pass


class Cart:
    def __init__(self):
        self.items = []

    def add(self, name, price, qty=1):
        if price < 0:
            raise ValueError("price must be >= 0")
        self.items.append((name, price, qty))

    def total(self):
        if not self.items:
            raise EmptyCartError("cart is empty")
        return sum(p * q for _, p, q in self.items)
```

test_cart.py

```
import pytest
from cart import Cart, EmptyCartError


def test_total_sums_items():
    cart = Cart()
    cart.add("book", 12000, 2)
    assert cart.total() == 24000


def test_empty_cart_raises():
    with pytest.raises(EmptyCartError):
        Cart().total()
```

HANDS-ON

실습 순서

STEP 01 pytest 설치와 확인

실행 환경을 갖춥니다.

```
pip install pytest
pytest --version
```

확인 포인트

- [ ] pytest 8.x 처럼 버전이 출력된다

> 가상환경을 쓴다면 활성화한 상태에서 설치하세요. **pip list** 로 어디에 깔렸는지 확인하는 습관을 들이면 좋습니다.

STEP 02 대상 코드와 첫 테스트 작성

테스트할 코드와 테스트를 만듭니다.

```
# 위 '실습용 파일'의 cart.py 와 test_cart.py 를 같은 폴더에 만듭니다
pytest
```

확인 포인트

- [ ] collected 2 items 가 나온다
- [ ] 점 두 개(..)와 2 passed 가 출력된다

> assert 한 줄이면 충분합니다. unittest 처럼 assertEqual 같은 메서드를 외울 필요가 없습니다.

STEP 03 수집 규칙 어겨 보기 — 오늘의 핵심 (1)

이름 하나로 테스트가 사라지는 것을 확인합니다.

```
# 실험 A · 파일 이름 바꾸기
mv test_cart.py cart_tests.py
pytest
mv cart_tests.py test_cart.py

# 실험 B · 함수 이름 바꾸기
# test_total_sums_items → check_total_sums_items 로 수정
pytest

# 실험 C · 어떤 것이 수집되는지만 보기
pytest --collect-only
```

확인 포인트

- [ ] A: collected 0 items · 오류는 나지 않는다
- [ ] B: collected 1 item — 나머지 하나가 조용히 사라졌다
- [ ] C: 수집된 테스트 목록이 그대로 출력된다

> **--collect-only 를 기억하세요.** "분명히 썼는데 안 돈다" 싶을 때 가장 먼저 칠 명령입니다.
> 이름을 바꿔 테스트를 끄지 마세요. 실수로 영영 잊힙니다. 끄려면 **@pytest.mark.skip(reason="...")** 을 쓰세요.

STEP 04 일부러 실패시키고 리포트 읽기 — 오늘의 핵심 (2)

실패 메시지의 각 줄이 무엇을 뜻하는지 익힙니다.

```
# test_cart.py 의 기대값을 틀리게 바꿉니다
assert cart.total() == 0
pytest
```

확인 포인트

- [ ] > 표시가 붙은 줄이 실패한 바로 그 문장이다
- [ ] E assert 24000 == 0 — 왼쪽이 실제, 오른쪽이 기대값이다
- [ ] + where 24000 = <Cart>.total() — 그 값이 어디서 나왔는지 알려 준다
- [ ] 맨 아래 test_cart.py:7 은 파일과 줄 번호다

> 다른 언어의 테스트 도구와 달리 pytest는 **assert 문을 다시 써서 양쪽 값을 자동으로 보여 줍니다.** 그래서 메시지를 직접 적을 필요가 거의 없습니다. 확인 후 기대값을 원래대로 되돌리세요.

STEP 05 예외를 검증하기

실패해야 정상인 경우를 테스트합니다.

```
# test_cart.py 에 추가
def test_negative_price_rejected():
    cart = Cart()
    with pytest.raises(ValueError, match="price must be"):
        cart.add("book", -100)

pytest -v
```

확인 포인트

- [ ] 3 passed 가 된다
- [ ] match 로 메시지 내용까지 확인할 수 있다

> **pytest.raises 없이** 예외가 나는 코드를 그냥 부르면 테스트 자체가 에러로 끝납니다. 예외가 나야 정상인 상황은 반드시 이 형태로 감싸세요.

STEP 06 실행 옵션 비교하기

옵션에 따라 무엇이 실행되는지 확인합니다.

```
# 두 개를 일부러 실패하게 만들어 두고 아래를 차례로 실행
pytest
pytest -v
pytest -q
pytest -k "total"
pytest -x
pytest --lf
```

확인 포인트

- [ ] 옵션 없음: 점(.)과 F 로만 표시된다
- [ ] -k: deselected 개수가 함께 표시된다
- [ ] -x: 첫 실패에서 멈추고 뒤 테스트는 실행되지 않는다
- [ ] --lf: 직전에 실패한 것만 다시 돈다

> **--lf 는 직전 실행 기록이 있어야 동작합니다.** 고치는 중에는 **pytest --lf -x** 조합이 가장 빠릅니다.

STEP 07 프로젝트 구조 정리

실무에서 쓰는 폴더 구조로 옮깁니다.

```
mkdir -p src tests
mv cart.py src/
mv test_cart.py tests/
pytest

# ModuleNotFoundError 가 나면 pyproject.toml 에 아래를 추가
[tool.pytest.ini_options]
pythonpath = ["src"]

pytest
```

확인 포인트

- [ ] 처음에는 ModuleNotFoundError: No module named 'cart' 가 난다
- [ ] pythonpath 설정 후 정상 실행된다

> 이 오류는 거의 모든 초심자가 한 번은 만납니다. pytest는 **테스트 파일이 있는 곳**을 기준으로 import 하므로, 소스가 다른 폴더에 있으면 경로를 알려 줘야 합니다.

STEP 08 skip 과 xfail

테스트를 끄는 올바른 방법을 익힙니다.

```
import sys

@pytest.mark.skip(reason="결제 모듈 리팩터링 중")
def test_pay(): ...

@pytest.mark.skipif(sys.platform == "win32", reason="리눅스 전용")
def test_permission(): ...

@pytest.mark.xfail(reason="알려진 버그, 티켓 #123")
def test_known_bug(): ...

pytest -v
```

확인 포인트

- [ ] SKIPPED, XFAIL 로 표시되며 결과에 남는다
- [ ] 이유가 함께 출력된다

> 이름을 바꿔 끄는 것과 결정적으로 다른 점은 **결과에 계속 보인다는 것**입니다. 잊히지 않습니다.

TROUBLESHOOTING

자주 만나는 오류

증상	원인	해결
`collected 0 items`	파일·함수 이름이 규칙에 안 맞음	pytest --collect-only 로 확인 · test_ 로 시작하는지 점검
`ModuleNotFoundError: No module named ...`	소스 폴더가 import 경로에 없음	pyproject.toml 에 pythonpath 설정 또는 pip install -e .
`fixture 'x' not found`	존재하지 않는 인자를 테스트 함수에 씀	오타 확인 · 4교시에서 fixture 를 배웁니다
예외 테스트가 `error` 로 끝남	pytest.raises 로 감싸지 않음	with pytest.raises(...) 안에서 호출
테스트가 통과했는데 미덥지 않음	assert 를 빠뜨렸거나 항상 참인 조건	실패하도록 한 번 바꿔 보고 정말 빨간불이 나는지 확인

REVIEW

복습 체크리스트 — 답할 수 있으면 통과

- [ ] 테스트 파일 이름 규칙 두 가지는 무엇인가요?
- [ ] collected 0 items 가 나왔을 때 가장 먼저 칠 명령은?
- [ ] 실패 리포트에서 > 표시와 E 표시는 각각 무엇인가요?
- [ ] 예외가 발생해야 정상인 코드는 어떻게 테스트하나요?
- [ ] 테스트를 잠시 끄고 싶을 때 이름을 바꾸면 안 되는 이유는?

한 줄 정리

테스트는 통과할 때보다 실패할 때 값어치가 있습니다. 실패 리포트를 읽을 줄 알면 나머지는 따라옵니다.

다음 시간 → 4교시 — 준비 코드를 나누고, 케이스를 늘리고, 외부를 끊어내기

WORKSHEET

막힌 STEP 번호 · 화면에 나온 오류 메시지 · 물어볼 것